some function is invoked and outputs are created. For each set of inputs (test cases) you specified in step 2, specify the expected outputs as they are characterized in the behavioral model.

4. **Execute the test cases.** Test cases can be executed manually, or a test script can be created for use by an automated testing tool.
5. **Compare actual and expected results and take corrective action as required.**

MBT helps to uncover errors in software behavior, and as a consequence, it is extremely useful when testing event-driven applications such as context-aware MobileApps.

21.11 Testing Virtual Environments

It is virtually impossible for a software developer to foresee how the customer will actually use a program. Instructions for use may be misinterpreted; strange combinations of input actions may be used; feedback that seemed clear to the tester may be unintelligible to a user in the field. User experience designers are very aware of the importance of getting feedback from actual users early in the prototyping process to avoid creating software that users dislike.

Acceptance tests are a series of specific tests conducted by the customer in an attempt to uncover product errors before accepting the software from the developer. Conducted by the end user rather than software engineers, acceptance testing can range from an informal "test drive" to a planned and systematically executed series of scripted tests.

When a software product is built for one customer, it is reasonable for that person to conduct a series to validate all requirements. If software is a virtual simulation or game developed as a product to be used by many customers, it is impractical to allow each user to perform formal acceptance tests. Most software product builders use a process called alpha and beta testing to uncover errors that only end users seem able to find.

The *alpha test* is conducted at the developer's site by a representative group of end users. The software is used in a natural setting with the developer "looking over the shoulder" of the users and recording errors and usage problems. Alpha tests are conducted in a controlled environment.

The *beta test* is conducted at one or more end-user sites. Unlike alpha testing, the developer generally is not present. Therefore, the beta test is a "live" application of the software in an environment that cannot be controlled by the developer. The customer records all problems (real or imagined) that are encountered during beta testing and reports these at regular intervals. As a result of problems reported during beta tests, the developer makes modifications and then prepares for release of the software product to the entire customer base.

21.11.1 Usability Testing

Usability testing evaluates the degree to which users can interact effectively with the app and the degree to which the app guides users' actions, provides meaningful

feedback, and enforces a consistent interaction approach. Rather than focusing intently on the semantics of some interactive objective, usability reviews and tests are designed to determine the degree to which the app interface makes the user's life easy.[7]

Developers contribute to the design of usability tests, but in general the tests are conducted by end users. Usability testing can occur at a variety of different levels of abstraction: (1) the usability of a specific interface mechanism (e.g., a form) can be assessed, (2) the usability of a complete virtual interface (encompassing interface mechanisms, data objects, and related functions) can be evaluated, or (3) the usability of the complete virtual-world application can be considered.

The first step in usability testing is to identify a set of usability categories and establish testing objectives for each category. The following test categories and objectives (written in the form of a question) illustrate this approach:[8]

Interactivity. Are interaction mechanisms (e.g., pull-down menus, buttons, widgets, inputs) easy to understand and use?

Layout. Are navigation mechanisms, content, and functions placed in a manner that allows the user to find them quickly?

Readability. Is text well written and understandable?[9] Are graphic representations easy to understand?

Aesthetics. Do layout, color, typeface, and related characteristics lead to ease of use? Do users "feel comfortable" with the look and feel of the app?

Display characteristics. Does the app make optimal use of screen size and resolution?

Time sensitivity. Can important features, functions, and content be used or acquired in a timely manner?

Feedback. Do users receive meaningful feedback to their actions? Is the user's work interruptible and recoverable when a system message is displayed?

Personalization. Does the app tailor itself to the specific needs of different user categories or individual users?

Help. Is it easy for users to access help and other support options?

Accessibility. Is the app accessible to people who have disabilities?

Trustworthiness. Are users able to control how personal information is shared? Does the app make use of personal information without user permission?

A series of tests is designed within each of these categories. In some cases, the "test" may be a visual review of the app screen displays. In other cases interface semantics tests may be executed again, but in this instance usability concerns are paramount.

---

7 The term *user-friendliness* has been used in this context. The problem, of course, is that one user's perception of a "friendly" interface may be radically different from another's.

8 For additional information on usability, see Chapter 12.

9 The FOG Readability Index and others may be used to provide a quantitative assessment of readability.

FIGURE 21.3
Qualitative assessment of usability



As an example, we consider usability assessment for interaction and interface mechanisms. The following is a list of interface features that might be reviewed and tested for usability: animations, buttons, color, control, graphics, labels, menus, messages, navigation, selection mechanisms, text, and HUDs[10] (heads-up user displays). As each feature is assessed, it is graded on a qualitative scale by the users who are doing the testing. Figure 21.3 shows a possible set of assessment "grades" that can be selected by users. These grades are applied to each feature individually, to a complete app screen display, or to the app as a whole.

21.11.2 Accessibility Testing

Accessibility testing is the process of verifying the degree to which all people can use a computer system regardless of any user's special need. The special needs most commonly considered for computer system accessibility are: visual, hearing, movement, and cognitive impairments [Zan18]. Many of these special needs evolve as people get older. As a profession, virtual environment development has not done a good job of providing access systems with rich graphical interfaces that rely heavily on touch interactions [Dia14]. The problems merely shift with a switch to voice-activated

10 Mobile apps and games frequently provide graphical displays containing user status, system messages, navigation date, and menu choices as part of the device screen display or HUD.

personal assistants like Alexa® or Siri®. Just imagine trying to operate your smartphone without using all these senses: sight, hearing, touch, or speech.

We discussed guidelines[11] for designing accessible software products in Chapter 13. An effective design strategy should ensure that all important interactions with the user be presented using more than one information channel. A few examples of the areas of focus for accessibility testing follow [Zan18] [Dia14]:

- Ensure that all nontext screen objects are also represented by a text-based description.
- Verify that color is not used exclusively to convey information to the user.
- Demonstrate that high contrast and magnification options are available for elderly or visually challenged users.
- Ensure that speech input alternatives have been implemented to accommodate users that may not be able to manipulate a keyboard, keypad, or mouse.
- Demonstrate that blinking, scrolling, or auto content updating is avoided to accommodate users with reading difficulties.

It is likely that mobile, cloud-based software will come to dominate many things that users need to accomplish on a day-to-day basis (e.g., banking, tax preparation, restaurant reservations, trip planning) and as a consequence, the need for accessible software products will only grow. Along with expert review and automated tools to assess accessibility, a thorough accessibility testing strategy will help to ensure that every user, no matter their challenges, will be accommodated.

21.11.3 Playability Testing

Playability is the degree to which a game or simulation is fun to play and usable by the user/player and was originally conceived as part of the development of video games. Game playability is affected by the quality of the game: usability, storyline, strategy, mechanics, realism, graphics, and sound. With the advent of virtual/augmented reality simulations whose intention is to provide entertainment or learning opportunities (e.g., such as simulated troubleshooting), it makes sense to use playability testing as part of the usability testing for a virtual environment created by MobileApp [Vel16].

Expert review can be used as part of the playability testing, but unless expert users are your target user group you may not get the feedback you need to have your MobileApp succeed in the marketplace. Expert review should be supplemented by playability tests conducted by representative end users, as you might do for a beta or acceptance test. In a typical play test, the user might be given general instructions on using the app and the developers would then step back and observe players use of the game without interruption. The players may be asked to complete a survey on their experience once they are done with the play test [Hus15].

The developers might record the play session or simply take notes on what they observe. The developers are looking for places in the play session where the player does

11 Here is an example of a software accessibility checklist used by the United States Department of Justice: https://www.justice.gov/crt/software-accessibility-checklist.

not know what to do next (this is usually marked by a sudden halt in the player's actions). Developers should note where the player is in the app work flow when this event happens. When the play test has ended, the developers might discuss why the player got stuck and how the player got herself unstuck (it she did). This suggests that playability testing might be helpful in assessing the accessibility of a virtual environment as well.

21.12 Testing Documentation and Help Facilities

The term *usability testing* conjures up images of large numbers of test cases prepared to exercise computer programs and the data that they manipulate. But errors in help facilities or online program documentation can be as devastating to the acceptance of the program as errors in data or source code. Nothing is more frustrating than following a user guide or help facility exactly and getting results or behaviors that do not coincide with those predicted by the documentation. It is for this reason that documentation testing should be an important part of every software test plan.

Documentation testing can be approached in two phases. The first phase, technical review (Chapter 16), examines the document for editorial clarity. The second phase, live test, uses the documentation in conjunction with the actual program.

Surprisingly, a live test for documentation can be approached using techniques that are analogous to many of the black-box testing methods discussed earlier. Graph-based testing can be used to describe the use of the program; equivalence partitioning and boundary value analysis can be used to define various classes of input and associated interactions. MBT can be used to ensure that documented behavior and actual behavior coincide. Program usage is then tracked through the documentation.

INFO



Documentation Testing

The following questions should be answered during documentation and/or help facility testing:

- Does the documentation accurately describe how to accomplish each mode of use?
- Is the description of each interaction sequence accurate?
- Are examples accurate?
- Are terminology, menu descriptions, and system responses consistent with the actual program?
- Is it relatively easy to locate guidance within the documentation?
- Can troubleshooting be accomplished easily with the documentation?
- Are the document's table of contents and index robust, accurate, and complete?
- Is the design of the document (layout, typefaces, indentation, graphics) conducive to understanding and quick assimilation of information?
- Are all software error messages displayed for the user described in more detail in the document? Are actions to be taken as a consequence of an error message clearly delineated?
- If hypertext links are used, are they accurate and complete?
- If hypertext is used, is the navigation design appropriate for the information required?

The only viable way to answer these questions is to have an independent third party (e.g., selected users) test the documentation in the context of program usage. All discrepancies are noted, and areas of document ambiguity or weakness are defined for potential rewrite.

21.13 SUMMARY

The goal of MobileApp testing is to exercise each of the many dimensions of software quality for mobile applications with the intent of finding errors or uncovering issues that may lead to quality failures. Testing focuses on the quality elements such as content, function, structure, usability, use of context, navigability, performance, power management, compatibility, interoperability, capacity, and security. It incorporates reviews and usability assessments that occur as the MobileApp is designed, and tests that are conducted once the MobileApp has been implemented and deployed on an actual device.

The MobileApp testing strategy exercises each quality dimension by initially examining "units" of content, functionality, or navigation. Once individual units have been validated, the focus shifts to tests that exercise the MobileApp as a whole. To accomplish this, many tests are derived from the user's perspective and are driven by information contained in use cases. A MobileApp test plan is developed and identifies testing steps, work products (e.g., test cases), and mechanisms for the evaluation of test results. The testing process encompasses several different types of testing.

Content testing (and reviews) focus on various categories of content. The intent is to examine errors that affect the presentation of the content to the end user. The content needs to be examined for performance issues imposed by the mobile device constraints. Interface testing exercises the interaction mechanisms that define the user experience provided by the MobileApp. The intent is to uncover errors that result when the MobileApp does not take device, user, or location context into account.

Navigation testing is based on use cases, derived as part of the modeling activity. The test cases are designed to exercise each usage scenario against the navigation design within the architectural framework used to deploy the MobileApp. Component testing exercises content and functional units within the MobileApp.

Performance testing encompasses a series of tests that are designed to assess MobileApp response time and reliability as demands on server-side resource capacity increase.

Security testing incorporates a series of tests designed to exploit vulnerabilities in the MobileApp or its environment. The intent is to find security holes in either the device operating environment or the Web services being accessed.

Finally, MobileApp testing should address performance issues such as power usage, processing speed, memory limitations, ability to recover from failures, and connectivity issues.

Navigation testing applies use cases, derived as part of the modeling activity, in the design of test cases that exercise each usage scenario against the navigation design. Navigation mechanisms are tested to ensure that any errors impeding completion of a use case are identified and corrected. Component testing exercises content and functional units within the MobileApp.

PROBLEMS AND POINTS TO PONDER

21.1. Are there any situations in which MobileApp testing on actual devices can be disregarded?

21.2. Is it fair to say that the overall mobility testing strategy begins with user-visible elements and moves toward technology elements? Are there exceptions to this strategy?